

ECE-320 Lab 2: Filtering and Control of a Wheel using On-Off and Proportional Control

Overview: In this lab we will first implement digital filters to remove noise from our measurements, and then implement an On-Off (bang-bang) controller and a proportional controller with a prefilter to control the speed of a wheel. We will be trying to design a controller without the use of a model of the wheel system. In the next lab we will be making a model of the wheel to use in designing a controller.

A DC motor makes the wheel spin. The speed at which the motor turns is controller by a pwm signal. This pwm signal is the sent through an external amplifier, since the dsPIC does not have the power to run the DC motor we are using.

The goals of this lab are:

- Learn to design and filter signals in discrete-time
- Learn to connect the dsPIC30F4011 to the VNH5019 motor driver board, and the VNH5019 motor driver board to the motor on our wheel system.
- Learn to connect the encoder from the wheel system to the dsPIC.
- Learn to change the values from the wheel encoder to radians/second.
- Learn to scale the input (reference signal).

Note that if your system stops or shuts itself off when trying to run the wheel system, you probably need larger capacitors across the input pins into the dsPIC. This is a very common problem.

Figure 1 shows the dsPIC30F4011 with the 40-PIN pinout. In what follows, you should not have power connected until you are done wiring each part.

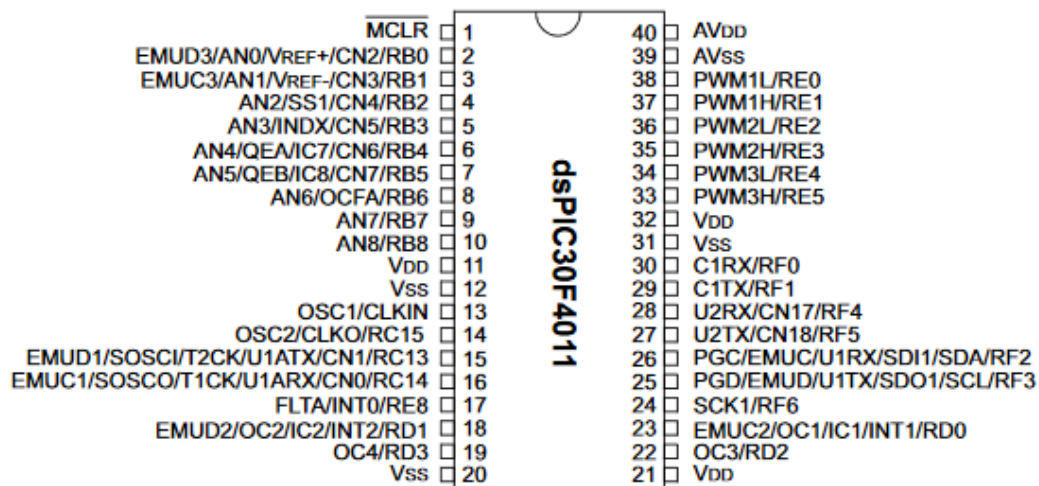


Figure 1. dsPIC30F4011 40-PIN pinout.

Part A Designing and Implementing a Discrete-time Filter

One thing you might have noticed is that the values from the pot sometimes seem to be very jittery. The speed of the system may also appear that way. In addition, the belt on the wheel system can slip which will cause strange spikes in the measured data.

One way to deal with this problem, in part anyway, is to lowpass filter the reference signal (the input) and the measured wheel speed signal (the output) to try to make them smoother. However, any practical filter is going to introduce a delay in the system. We want to be sure the delay is small, and, for us, the delay between the reference and speed signals are the same (we could model different delays, but that will complicate things more than necessary.)

Recall from ECE-180, that if we have the 3rd order discrete-time filter (or transfer function)

$$G(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2} + b_3 z^{-3}}{1 + a_1 z^{-1} + a_2 z^{-2} + a_3 z^{-3}} = \frac{B(z)}{A(z)} = \frac{\text{Output}(z)}{\text{Input}(z)}$$

we can cross multiply to get

$$\begin{aligned} \text{Output}(z) + a_1 z^{-1} \text{Output}(z) + a_2 z^{-2} \text{Output}(z) + a_3 z^{-3} \text{Output}(z) \\ = b_0 \text{Input}(z) + b_1 z^{-1} \text{Input}(z) + b_2 z^{-2} \text{Input}(z) + b_3 z^{-3} \text{Input}(z) \end{aligned}$$

Recall the inverse z -transform of $\text{Output}(z)$ is $\text{Output}[n]$. In addition, z^{-m} corresponds to a delay of m , so that $z^{-m} \text{Output}(z)$ corresponds to $\text{Output}[n - m]$. Applying this idea our equation above, we get

$$\begin{aligned} \text{Output}[n] + a_1 \text{Output}[n - 1] + a_2 \text{Output}[n - 2] + a_3 \text{Output}[n - 3] \\ = b_0 \text{Input}[n] + b_1 \text{Input}[n - 1] + b_2 \text{Input}[n - 2] + b_3 \text{Input}[n - 3] \end{aligned}$$

Note that for implementation, we might want to write this as

$$\begin{aligned} \text{Output}[n] = & b_0 \text{Input}[n] \\ & + b_1 \text{Input}[n - 1] - a_1 \text{Output}[n - 1] \\ & + b_2 \text{Input}[n - 2] - a_2 \text{Output}[n - 2] \\ & + b_3 \text{Input}[n - 3] - a_3 \text{Output}[n - 3] \end{aligned}$$

Also, for our implementation, the newest (most recent) data is located in $\text{Input}[0]$ and $\text{Output}[0]$, while the oldest data is located in $\text{Input}[n-3]$ and $\text{Output}[n-3]$. *You should not try to change this ordering! The output of the filter should be written to the 0th value (the first value) of the output.*

The first thing you need to do is implement an IIR filter using the ideas above. Open a new project in MPLABX and use the source code **check_IIR_filter.c**. There is a routine in this code where you are to implement an IIR filter (called, surprisingly, *filter*). You need to write this routine so it can handle a filter of any order N . There are also three test filters (and the correct responses).

Just before we filter the data, we are going to want to update the Input (Rin) and Output (Rout) vectors, so the newest elements will be the first ones (with an index of 0, since C starts indices at 0), and the oldest ones will be at the end of the array. The routine **update_array** accomplishes this, and is shown in Figure 2 for your reference.

```
void update_array( double arr[], int N)
{
    int k;
    for (k=N-2; k>=0; k--) arr[k+1] = arr[k];
}
```

Figure 2. Code that updates the arrays (shifts all the elements downward, so the newest elements in the first position, with index 0).

You should implement your filtering routine then compile and run it. The program will write to the screen (SecureCRT) the sample time (you don't care about this), the output of your filter routine, the correct output, and the error. The error should be very near zero (to at least three decimal places). Once you have debugged your filter routine and it seems to be working, comment out the section of code **Filter test1**, uncomment the section of code **Filter test2**, compile and run your code. Again, you should get an error of nearly zero or there is an error in your code. Once you fix this, comment out the section of code **Filter test2**, uncommnet **Filter test3**, compile and run your code. *Your final code should produce an error of nearly zero for all three of the test files.*

Now all we need is to determine how to get the coefficients (the a_i and b_i) before we implement our filter.

On the class Moodle site, there is a MATLAB file named **wheel_parameters.m**. This is a file that contains parameters we determine for our wheel system that will be used by our MATLAB/Simulink model of our system. You will be modifying the parameters in this file as you do the next few labs. The important section of this file is shown in Figure 3, where the filter is determined and the filter coefficients are found. *However, it is important for now and future labs that the values in this file match the corresponding values in the C-code you are using.*

```
%
% design a lowpass filter
%
Fs = 1/Ts; % sampling rate in Hz
nfilt = 3; % filter order
hp_freq = 3.0; % halfpower frequency

d = designfilt('lowpassiir','FilterOrder',nfilt, ...
    'HalfPowerFrequency',hp_freq,'SampleRate',Fs);
fvtool(d)
[B, A] = tf(d)
[z,p,k] = zpkm(d)
```

Figure 3. Determining a filter part of the wheel_parameters.m file.

You can change the filter order and half power frequency as you wish, but keep in mind that the larger the filter order, the more delay you are introducing in your system, which is generally not a good idea in a control system. You should run your system for a number of different values to find one that you feel works well (note that you will not be able to eliminate all of the wiggles in the data, just reduce them somewhat.)

The code for the remainder of this lab is on the Moodle site is called **wheel_modelling_and_control.c**. You need to download this file, and setup MPLABX to work with this file. At this point, there are many parts of this code that are commented out because we will use them as we go through the labs.

Run **wheel_parameters.m**, with the parameters used in Figure 3 Here we have decided to use a third order filter, but you can change that later if you wish. The values for the A and B vectors have been loaded into your C-file, and arrays for the input (Rin) and output (Rout) vectors have been sized and initialized to 0. This is all done in the piece of code shown in Figure 4, which is located just under the *main* declaration in the code. (It obviously does some other things to).

```
double Y, Yin[4] = {0.0, 0.0, 0.0, 0.0};
double Yout[4] = {0.0, 0.0, 0.0, 0.0};
double R, Rin[4] = {0.0, 0.0, 0.0, 0.0};
double Rout[4] = {0.0, 0.0, 0.0, 0.0};
double A[4] = {1.0, -1.1619, 0.6959, -0.1278}, B[4] = {0.0495, 0.1486,
0.1486, 0.0495};
int Nr =4, Ny=4;
```

Figure 4. Code that initializes the arrays and enters to filter coefficients.

In the code just under the PREFILTER heading (just below the MAIN LOOP) you will see the section of code shown in Figure 5. What you need to do is to implement the piece of code that does the filtering, the **filter** routine. The most recent filtered data should end up in Rout[0]. Note that this section of code also has some checks to be sure our final value of the reference signal, R, is in the correct range.

```
update_array(Rout, Nr );
update_array(Rin, Nr);
Rin[0] = AD_value;
filter(A, B, Rin, Rout, Nr); ← you need to make the filter routine
R = Rout[0];
R = max(0.0,R);
R = min(R,1023);
R = R* AD_scale;
```

Figure 5. Code used to filter the input (reference) signal.

To check your filter, once your code compiles, log the output (written to the screen of your terminal emulator, most likely SecureCRT) to a file named **filter_response**. You will probably find it easiest to do this part if you use Secure CRT to record the results. In order to record your results, go through the following steps in Secure CRT:

Select Edit-> Clear Screen and Scrollback

Select File-> Log Session. Name the file **filter_response**, and put it in the same place your other code for this lab is. You can change the name later if you want.

Run the program (microcontroller)

To stop the recording you again need to *Select File-> Log Session*.

The MATLAB script we are going to use is named **check_filter.m**, which you should be able to get from the class Moodle site. This script assumes that your response is written to a file named **filter_response**. It assumes the data written to the log file has the output in the order: (1) the current time (multiplied by 100), (2) the unfiltered reference signal, and (3) the filtered reference signal. If your C function does not write out the data in this order, you need to modify it. *As we go on, you will see that we usually want to print integers since converting and printing doubles to the screen is very time consuming for the microcontroller.*

In MATLAB, you first need to open your file **filter_response** and removed the first line and any strange characters, any data you know you do not want, and then close the file. You may need to edit **check_filter.m** to be sure it is reading in the correct file. The value of **t_fix** should be the first sample time of your data. This variable allows you to truncate the data (like removing the first 5 seconds), and then the program will subtract 5 from all of the times, so it appears to start at time 0. Now run **wheel_parameters.m** to be sure the A and B vectors are in the workspace, and be sure these values are the same you used in your C-code. Then run the microcontroller code and *move the pot continually to change the reference signal. You will want the input signal to be varying so you can see the filtering effect*. Run the microcontroller for 10-15 seconds.

When you stop the code run **check_filter.m**. This program first plots the unfiltered and filtered signals, and then runs the unfiltered signal through a Simulink model (**simulate_filter.slx**) and plots the unfiltered signal, your filtered signal, and the signal that has been run through the Simulink file implementing the same filter. Your filtered signal and the Simulink results should be the same, if they are not you need to fix your code.

Once your filtered signal matches that from Simulink, put a copy of this figure in your memo. Be sure you have run the system for at least 10 seconds, and be sure the input is varying. Once your filter results match those of the Simulink model, you should feel free to vary the filter if you wish.

Include a picture of the output of the **check_filter** program, showing that your filter matches that of the Simulink model. Also include the filter order (*nfilt*) and the half_power frequency (*hp_freq*) from **wheel_parameters.m**.

At this point, you may notice that the filtered signal is actually delayed (it lags the unfiltered signal.) In many signal processing applications this may not be a problem, but in a control systems it can be. Since we will be trying to control the speed of a wheel system for the next few labs, and the response of the system is fairly slow, it will not affect our system much. However, it is very important that the output signal is filtered the same way as the input signal, since we will be comparing them to see how far off we are.

In the section of code under the FEEDBACK (H) section, you need to implement a filter for the output (Y) signal. Put the output filtering **after** the variable *speed* is determined and the checks to be sure it has not varied too much from the last value of speed. Be sure to update the arrays just before you filter, and your input and output arrays should be *Yin* and *Yout*. The most recent value should be *Yin[0]* and is the current (unfiltered) *speed* of the wheel. You also need to be sure the resulting values for *Yout[0]* are positive or zero. However, do not apply any other limits to *Yout[0]*. After filtering, the most recent filtered signal should be set equal to *Y*. **Do not** multiply *Y* (or *Yout*) by *AD_Scale*.

Part B: Connecting the Wheel Encoder

Now we want to be able to determine the speed of rotation of the wheel in the wheel system. To do this we need to be able to connect the wheel encoder to our dsPIC, and then (later) we need to be able to convert the values from this encoder into the units of radians/second. Figure 6 shows the encoder connected to the dsPIC. **Vcc** is connected to the 5 volt line, **Gnd** is connected to ground, **A** is connected to **QEA** (pin 6), and **B** is connected to **QEB** (pin 7). QE(A or B) stands for **Q**uadrature **E**ncoder channel **A** or **B**. Note that we will not be using **LED3** (pin 35), so you can disconnect it.

Part C: Connecting the VNH5019 Motor Driver Carrier—the amplifier

For this part you will need to checkout the VNH5019 motor driver, which is shown schematically in Figure 7. Note that we will only use a few of the pins on the board since we only want our wheel to go in one direction.

On the second half of your circuit board, you want to first connect the 5 volt rail and the ground rail to the corresponding rails on the original breadboard (the one with your dsPIC on it). Then connect **INA** (pin 1) and **VDD** (pin 7) to the 5 volt rail, connect **INB** (pin 6) and the ground (pin 8) just above the **VOUT** to the ground rail, and connect the **PWM** (pin 3) to **PWM3H** (pin 33) on the dsPIC.

Next connect your wheel system to the VNH5019, the red wire should connect to **OUTA** (pin 14), and the black wire should connect to **OUTB** (pin 13). The 12 volt power supply should connect to **VIN** (pin11) and the common ground **GND** to pin 12. Pin 11 should come from the positive pin on the barrel

connected to the power supply (this pin also is connected to the OKI-78SR power regulator.) Pin 12 should be a common ground rail you are using for the left side of the board. We need to be sure all of our “grounds” are connected together.

Figure 8 shows a schematic of the wiring for this board, and figure 9 shows a picture of the final system all wired together (however, the USB wire has been removed for clarity).

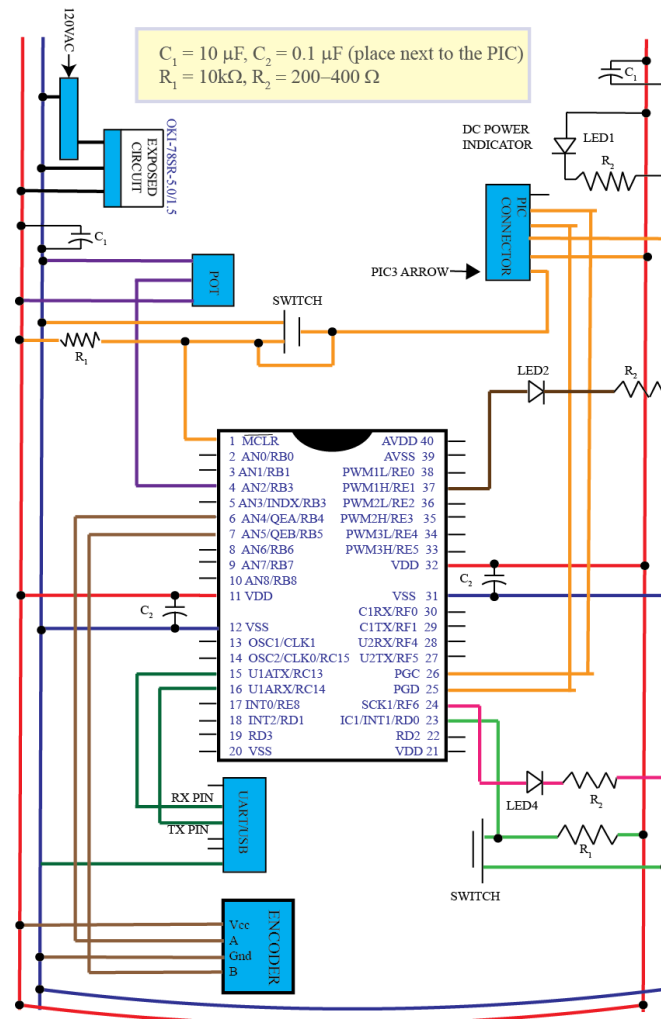


Figure 6. Connecting the encoder.

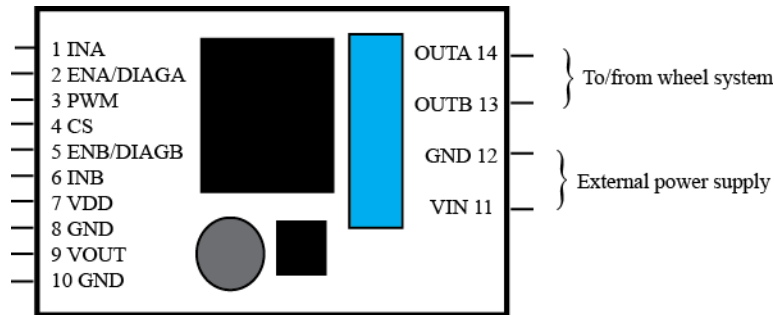


Figure 7. Schematic of the VNH5019 motor carrier board.

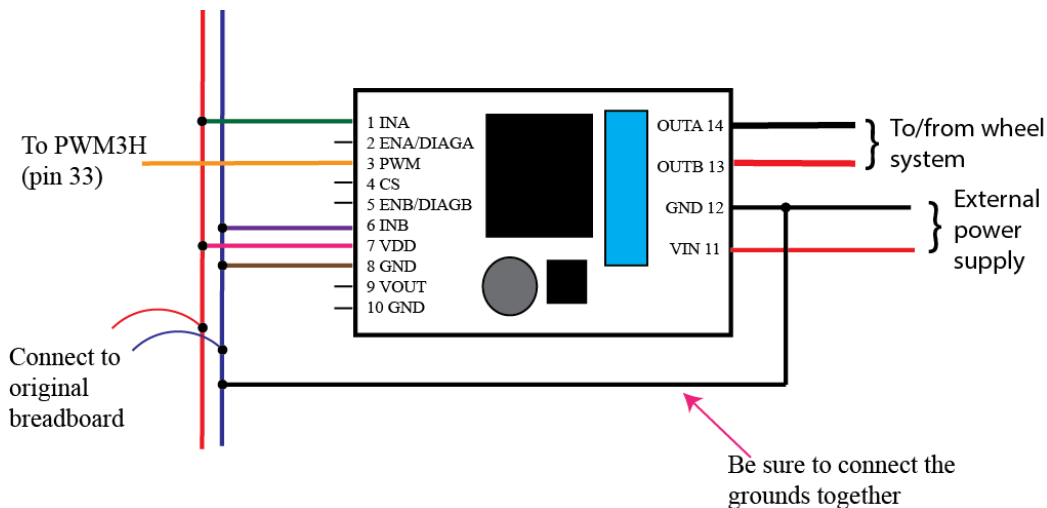


Figure 8. Schematic of the connections for the VNH5019 motor driver. The positive power for the motor (the external power) comes from the positive output of the barrel attached to the power supply.

Three important things to keep in mind as you do the remainder of this lab:

- *Compile and run all of the code in DEBUG mode. This will avoid the program starting again when you try to stop it.*
- *Use your shut off switch (external interrupt) to shut off the motor/stop the wheel. This actually stops the output, and puts the program in an infinite loop. You need to separately stop the program, or restart it.*
- *You will then need to select the MPLAB stop button to actually stop the program*
- *Unless you are told otherwise turn the pot fairly slowly.*

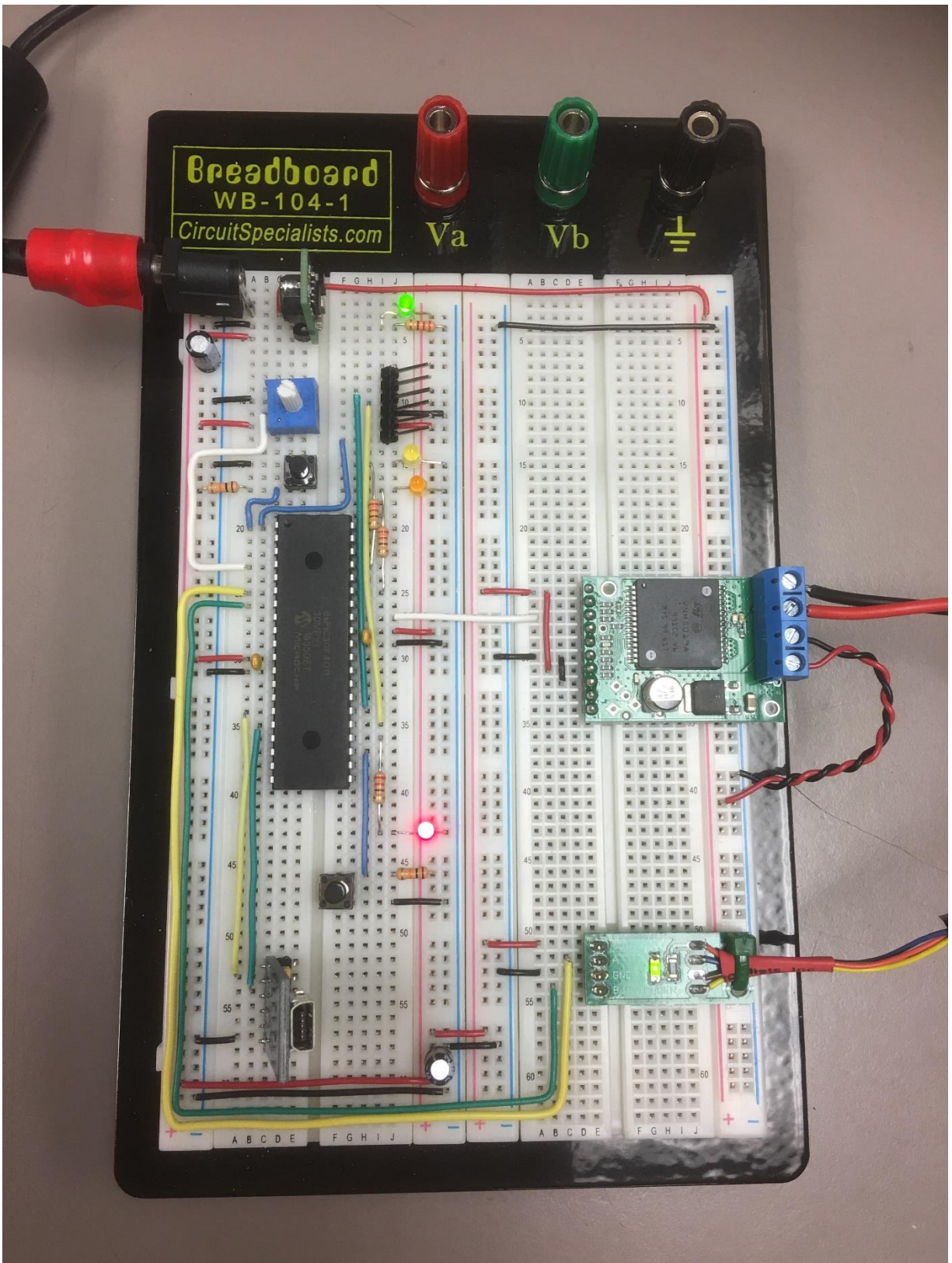


Figure 9. An example of the final circuit. Note the USB and PICkit3 are not connected.

PART D: Determining Initial Scaling

Now we will start to determine some of our parameters we need to control the speed of the motor. Note that the (filtered) reference signal is just the A/D value read in from the pot. We need to determine the scaling so the value read from the pot can be interpreted as the wheel speed in radians/second.

Near the end of the code, in the PRINT OUTPUT section, you need to modify your code so the reference signal `R` is converted to an integer with no other scaling, and then `Y` is multiplied by 100 and then converted to an integer. The variable `int_R` and `int_Y` have already been declared. We are doing this because the slowest part of our whole system is communicating over the UART, and writing integers takes a lot less time. You should print the variables in the order: time, reference signal, and the output.

In the CONTROL EFFORT CONVERSION part, you need to **uncomment** the code that runs the PWM (uncomment the command `SetDCMPWM...`) if this has not already been done.

You will probably find it easiest to determine the scaling if you use Secure CRT to record the results.

With the system at rest start the program and slowly turn the pot so the motor is spinning. The motor will not start for low input values so do not be alarmed if nothing happens at first. Continue to turn the pot slowly until the motor starts to spin the wheel. Turn the pot so the motor is spinning at less than or equal to 30 rad/sec (remember this will look like 3000 radians/sec because of the scaling.) *If the speed of the wheel is negative, reverse the polarity of the power wires to the motor.* Let the motor come to a reasonably steady value (it will likely keep getting faster). Wait until the motor speed is fairly constant and the input reading from the pot is also fairly constant. Record the reading from the pot and the speed of the motor. Do this for three different positions of the pot where the steady state speed is less than 80 rad/sec. *Stop the wheel by using your shut off /external interrupt switch! You can then restart the system without recompiling it.*

We now want to determine the proportionality constant between the A/D value read from the pot and the motor speed in rad/sec. That is, we want to convert from the A/D value read from the pot to the speed of the wheel in radians/sec. We want to use the real speed of the wheel, so you need to use the speed written to the screen and divide by 100. Compute the three scaling factors and average them (typically, this is between approximately 0.1 and 0.25).

Change the parameter **AD_scale** (in your code) to this value, and then scale the value of `int_R` so it is `R` multiplied by 100. At this point all of our outputs should be integers and have a value 100 times larger than their true values.

Run the system again at various input values. *It is usually best if you turn the pot to a value, and then let go of it until you want to change the value.* It may take a while for the system to reach steady state, and to reach lower speeds like 30 rad/sec you may have to start at a higher speed and work your way down to the desired speed. You will also not be able to get very large speeds either. Compare your reference

input to the actual output. They should be fairly close for some speeds but they will not be exact. For other speeds they may be off by a significant amount. ***This is one from of open loop control. We are not comparing the reference (input) speed to the actual (output) speed to correct any errors.***

Once your system is working reasonably well, run it again and record your output in a file named **results_open_loop**. You should start the system with a reference input of 0 radians/sec (from the pot) and then vary the input. Now run the MATLAB program (from the class Moodle site) **plot_results.m**. It is very likely that you will need to open the file **results_open_loop** in MATLAB and eliminate some weird characters and data you are not interested in at the beginning of the file. *Include this figure in your memo.* Be sure to use your interrupt (stop) button to stop the system, if you try and do it with MPLABX your motor may behave strangely. Figure 10 shows an example of open loop control, and it does not seem to work very well. Note that we are only controlling the speed of our wheel in one direction. If we want to slow down all we can do is coast.

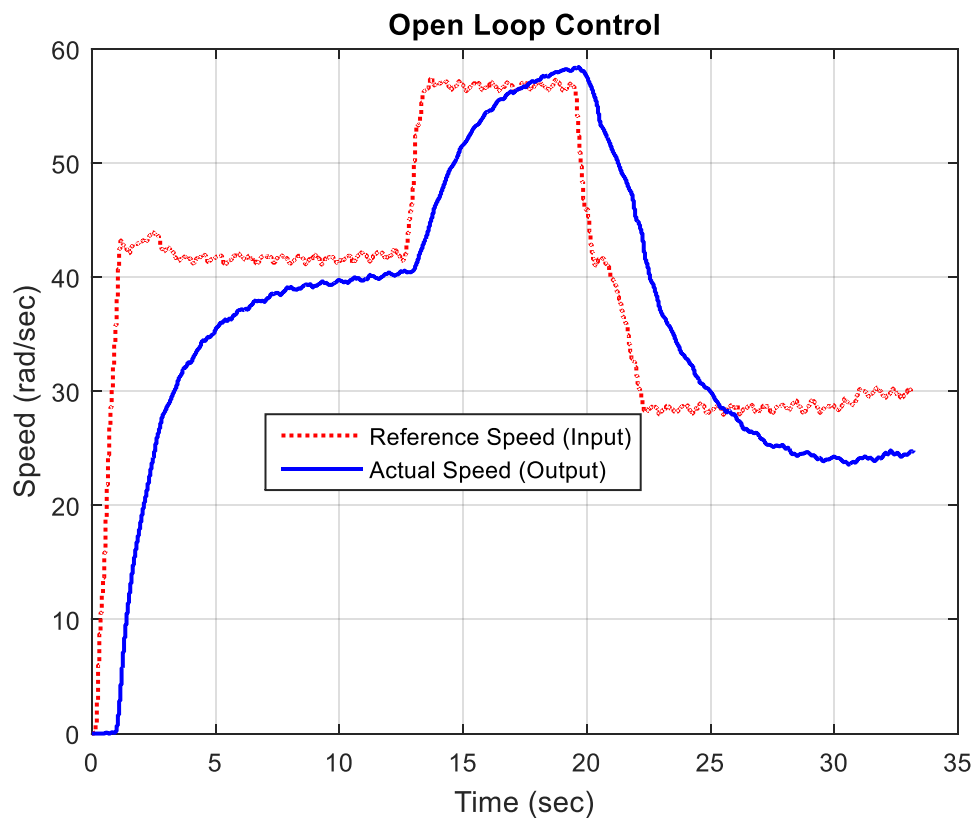


Figure 10. An example of open loop control.

At this point we should go over (briefly) how we are converting the input signal to a pwm signal that drives the motor.

We have basically (ignoring the filtering), the reference signal

$$R = AD_value * AD_scale,$$

where AD_value is the value read from the pot (between 0 and 1023). We then set (for our open loop control) our control effort equal to the reference signal, so

$$u = R.$$

Finally, we have to convert our control effort back to something our motor understands, which is in terms of the duty cycle of the pwm signal. This is where we use the variable **convert_to_duty**, which is

$$\text{convert_to_duty} = \text{MAX_DUTY}/1023$$

Finally we have the code

$$u = u * \text{convert_to_duty} / AD_scale$$

and if we write out all of the expressions, we get

$$u = R * (\text{MAX_DUTY}/1023) / (R/AD_value) = (AD_value/1023) * \text{MAX_DUTY}$$

Hence we are determining the fraction of the maximum duty cycle based on the value read from the pot (between 0 and 1023). When we actually implement a control we are change the value of the “reference” signal here, the control signal may be larger or smaller than the actual reference signal, but it is always a number between 0-1023 which when divided by 1023, which produces a fraction of the maximum duty cycle we want.

PART E: On-Off (Bang-Bang) Control

We will now try our first feedback controller, an on-off or bang-bang controller. This type of controller is either on or off (hence the name) and is usually used in systems with a large inertia (such as a furnace) so they do not react too poorly to suddenly being turned on. You will note when you do this that the wheel system shakes quite a lot, and this is hard on the belt.

In the ERROR computation part of the code, you need to **comment out** the line

$$\text{error} = R;$$

which is used for open loop control, and **uncomment** the line

$$\text{error} = R - Y;$$

which is used for feedback control.

If the error signal is positive, then the reference (input) signal is larger than the speed the motor is currently spinning at (the output signal) and we need to motor to turn on. If the error is negative, then the motor is spinning faster than the desired speed and we need to let the wheel coast down to the desired speed.

In the CONTROLLER section of the code **comment out** the code

```
u = u*convert_to_duty/AD_scale; // convert back to a pwm signal
```

and then just after this (in the CONTROL EFFORT CONVERSION section) add the code

```
if (error > 0.0) u = 0.05*MAX_DUTY;

else u = 0.0;
```

You should put this code just before the range of u is checked (u needs to be positive and less than MAX_DUTY). Note that when the motor is “on” we are only setting the control signal to a twentieth of its maximum allowed value (MAX_DUTY). Compile your code and run it. Increase the reference speed (via the pot) until your wheel is spinning pretty smoothly. *Mostly likely though, your motor just hums and does nothing since the input signal is not large enough to overcome its inertia.* Note however, that this is often what one does with a control system when you do not have a model—*start with a small control signal and gradually increase it.*

Now try setting $u = 0.2*MAX_DUTY$;

Remember the pot determines the reference speed you want the wheel to spin at. Starting at zero, you will run your system so there are two plateaus, as shown in Figure 11. One of the plateaus should be less than 15 rad/sec, and the other between 20 and 30 radians/sec. Note that because we are filtering the output signal, it is not reacting very quickly. If we did not filter this signal, the speed of the motor would change much more dramatically, and this is not good for the motor system. Another thing you will probably noticed is that you cannot make your wheel spin very fast. This is because we are seriously limiting the input signal to the motor. You should try to run the system for less than 30 seconds.

You should store (log) your data to **results_on_off**, and put it in the same place your C-code for this lab is. You can change the name later if you want, and you may want to change it to keep track of the different experiments you are running. Be sure to change the name in the MATLAB file **plot_results.m** and change the title for your figure. Run this MATLAB code and *put this figure in your memo.*

The next thing we are now going to explore is the maximum motor speed allowable as we change the maximum control effort (scaling the maximum control signal). With the scaling at $u = 0.2*MAX_DUTY$, start the system and determine the approximate maximum speed you can get the motor to go. (You do not need to log this; you can just read it off the output screen.) Let the motor run for about 10 seconds, it will have increased speed a bit as it goes on, but this is close enough. Do the same thing for scaling $0.3*MAX_DUTY$, $0.4*MAX_DUTY$, up to $0.7*MAX_DUTY$ and *make a table*

to include in your memo. You should notice two different things: (1) the motor seems to behave linearly for smaller control efforts, and (2) as we limit the control effort, we are limiting the maximum speed, even when the motor is behaving linearly.

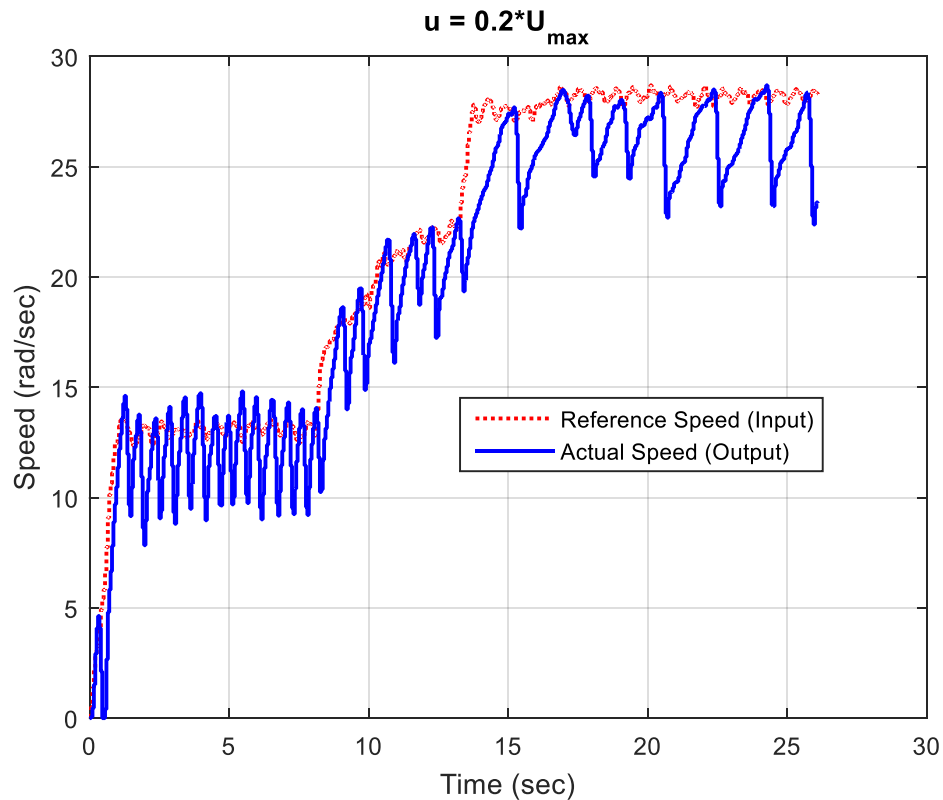


Figure 11. Results using a bang-bang controller with $u = 0.2 \cdot \text{MAX_DUTY}$.

Next, there is always some noise on the pot signal, so for now we are going to just code our reference signal so it will be always be the same.

In the PREFILTER section of code, **comment out** the line of code

```
R = Rout[0]*AD_scale;
```

and add the following lines:

```
if (time < 0.5) R = 0.0;
```

```
else if ((time >= 0.5) & (time < 8.5)) R = 10.0;
```

```
else if ((time >= 8.5) & (time < 20.5)) R = 25.0;
```

```
else if (time >= 20.5) R = 40.0;
```

Note that we are not filtering this signal!

Set $u = 0.20 * \text{MAX_DUTY}$, compile, run the code *and plot the results for your memo*, and then set $u = 0.5 * \text{MAX_DUTY}$, compile, run the code *and plot the results for your memo*. Do not let the system run more than around 30 seconds, and *be sure to change the title so the maximum control effort is in the title*. Your system may not reach 40 rad/sec so you can either change the maximum value of the reference signal, or just accept that your system cannot reach this value. *Include these two figures in your memo.*

You should have noticed from your table that as the maximum control effort increases, we could achieve faster speeds. As the most recent two plots demonstrate, as the control effort is set to a larger value the transition time between a lower speed and a higher speed is shorter. However, there is also a lot more variation in the speed of the wheel when we are trying to hold the speed constant. *This method of control is also very hard on this system, the belt will become loose very quickly.*

This leads to our next control method, **proportional control**, where the control effort is proportional to the error signal, or the difference between the reference (desired) signal and the current speed.

PART F: Proportional control

In the CONTROLLER section of the code, you need to **comment out** the code

```
if (error > 0.0) u = 0.5*MAX_DUTY;
```

```
else u = 0.0;
```

and **uncomment the code**

```
u = u*convert_to_duty/AD_scale;
```

You will need to declare the variable **kp** as a double (at the top of the main routine) and initialize it to 1.0. The control effort is then proportional to the error using this proportionality constant, so in the CONTROL section of code add

```
u = kp *error.
```

This should all be done immediately before the CONTROL EFFORT CONVERSION parts of the code.

Compile, download and run the program. You will probably have very poor results until the you get to the largest desired speed, and then your steady state error will be around one half of the expected steady state value.

Stop the system and change the value of **kp** to 5, and then to 10. For each of these runs log the results and use the MATLAB script **plot_results.m** to plot the results, *being sure to change the program to read the correct data file, and the title to indicate the correct value of kp*. Only run the system for 15-20 seconds. *You also probably want to name your log files something clever in case you screw up the plotting. Include these figures in your memo.*

You should notice that as **kp** gets larger, the steady state error gets smaller, and the speed of response when the reference signal changes decreases, but also the variation in the measured output signal is larger. You may also have a very larger percent overshoot. For now, we will concern ourselves with the steady state error.

PART G: Proportional control with scaling

Most likely your system did not equal the reference values in steady state, or not all of them. One way to try and get around this is to scale the input.

Set **kp** initially to some value between 5 and 10 that you think is reasonable, and run your code (and record the output). Plot your results using MATLAB, and use your plot to determine how much you need to scale your input (reference signal) by to get a steady state error of close to zero. Mostly likely you will not be able to find a single value of **scaling** that works for all of your plateau values (15, 25, and 40 rad/sec), so try and find one that works for the middle plateau.

Define a new (double) variable, **reference_scaling**, at the top of the main routine and initialize it to the value you just estimated, and modify the code within the PREFILTER section of code so after you have set all of the values of R, you have

```
R = R*reference_scaling;
```

Note that since we have scaled our reference input, for plotting purposes we want the original reference signal, so just before the print statement at the end of the code (in the PRINT OUTPUT section), you should modify **int_Rin** so it has the correct value of the reference input (multiplied by 100 still.)

Run your system with this value, record the output in an appropriately named file, and plot the results. You may need to use a little guess and check to get good answers. Once you have something you think is working well for the **middle plateau** **include this figure in your memo**. *Be sure the title contains the value you used for kp and Gpf (reference scaling).*

Note that most of you will not be able to get your outputs for all values of the reference signal, even using scaling. That is because our system is only approximately linear and our scaling is outside the feedback loop. We will be able to do much better when we use some form of integral control, but we will do that later. Also, you may have significant overshoot with your system. Scaling the input only helps with the steady state error.

PART H: Even More Proportional control with a prefilter

Make sure k_p and the value of the prefilter are set to the same values you determined in the previous part. Now we want the reference input to be from the pot rather than a set value, so modify the code appropriately (comment out the statements that set the values of R in the *if- else if* block, uncomment the code that sets R using the AD_value). *We need to remember that in a control system we are always trying to match the reference signal, step inputs that we have been using (and will continue to use) are just good testing models.*

Start the system at zero, and change the pot so the reference input is held stable until the system reaches that value, and then increased it again. Then decrease the reference signal in the same manner. This will result in a staircase sort of input. You only need to run your system for 20- 30 seconds or so, and should have at least three different steady state values (plateaus). An example of a typical result is plotted in Figure 12. Modify the MATLAB code (if needed) and run your system. *Put this figure in your memo.* How well did your system work? **Remember, for a control system the goal is to make the output equal to the input.**

Note that we can only coast down to lower speeds, so we do not expect the system to track very well as the speed is being reduced.

Be sure to write down your wheel number for next time!!

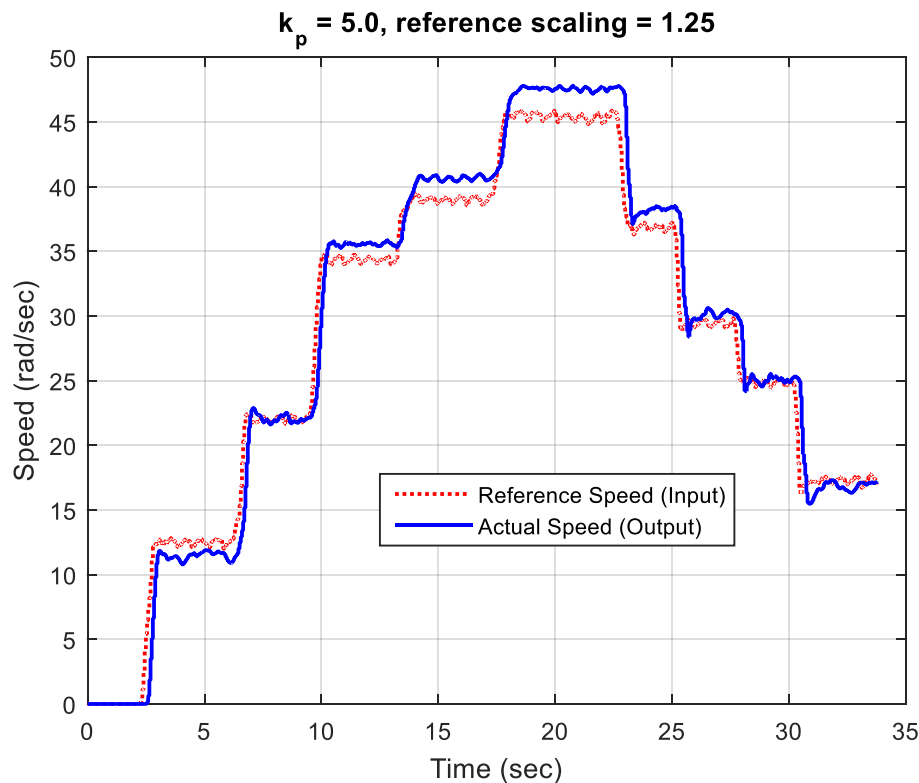


Figure 12. Proportional control with a prefilter for a staircase input